

Table of Contents

Wstęp	4
Rozdział 1. Kredyt i ocena zdolności kredytowej – ujęcie teoretyczne	4
1.1. Pojęcie kredytu i ryzyka kredytowego	4
1.2. Tradycyjne metody oceny zdolności kredytowej	4
1.2.1. Metody eksperckie i scoringowe	4
1.2.2. Regulacje prawne i standardy branżowe	4
1.3. Ograniczenia klasycznych metod scoringowych	4
1.4. Kierunki modernizacji oceny zdolności kredytowej	4
Rozdział 2. Uczenie maszynowe w zastosowaniach finansowych	4
2.1. Podstawy uczenia maszynowego	4
2.1.1. Taksonomia metod uczenia maszynowego	4
2.1.2. Kluczowe pojęcia: cechy, etykiety, nadmierne dopasowanie, walidacja	4
2.2. Zastosowanie uczenia maszynowego w sektorze finansowym	4
2.2.1. Przegląd zastosowań w zarządzaniu ryzykiem	4
2.2.2. Credit scoring oparty na metodach ML – przegląd literatury	4
2.3. Charakterystyka wybranych algorytmów	4
2.3.1. Sieci neuronowe z długą pamięcią krótkotrwałą (LSTM)	4
2.3.2. Lasy losowe (Random Forest)	4
2.3.3. Gradient Boosting – algorytm XGBoost.....	4
2.4. Porównanie algorytmów ML z klasycznymi metodami scoringowymi	5
2.5. Wyzwania etyczne i regulacyjne stosowania ML w ocenie kredytowej	5
Rozdział 3. Metodologia badań i projekt systemu	5
3.1. Cel i zakres badań	5
3.2. Postawione hipotezy badawcze.....	5
3.3. Struktura i charakterystyka danych dłużników	5
3.3.1. Źródła i sposób pozyskania danych	5
3.3.2. Opis zmiennych wejściowych i zmiennej docelowej	5
3.3.3. Preprocessing danych: czyszczenie, normalizacja, inżynieria cech	5
3.4. Projekt architektury systemu eksperymentalnego	5
3.4.1. Architektura ogólna i przepływ danych	5
3.4.2. Warstwa backendowa.....	5
3.4.3. Integracja modeli AI z systemem.....	5
3.4.4. Warstwa frontendowa.....	5

3.5. Obsługa wyjątków i scenariusze brzegowe	5
3.6. Narzędzia i środowisko technologiczne	5
Rozdział 4. Implementacja i trenowanie modeli	5
4.1. Podział zbioru danych i strategia walidacji	6
4.1.1. Podział na zbiór treningowy, walidacyjny i testowy	6
4.1.2. Techniki radzenia sobie z niezbalansowaniem klas (SMOTE, ważenie)	8
4.2. Implementacja modelu LSTM	10
4.2.1. Dobór architektury sieci i hiperparametrów	10
4.2.2. Proces trenowania i krzywe uczenia	12
4.3. Implementacja modelu Random Forest	14
4.3.1. Dobór liczby drzew i głębokości	14
4.3.2. Ważność cech (feature importance)	16
4.4. Implementacja modelu XGBoost	17
4.4.1. Strojenie hiperparametrów – Grid Search i Cross-Validation	18
4.4.2. Analiza wkładu cech (SHAP values)	20
4.5. Walidacja i testowanie modeli w środowisku eksperymentalnym	22
Rozdział 5. Analiza wyników i ocena modeli	24
5.1. Metryki oceny jakości modeli klasyfikacyjnych	24
5.1.1. Dokładność, precyzja, czułość, F1-Score	24
5.1.2. Krzywa ROC i pole pod krzywą (AUC-ROC)	24
5.1.3. Macierz pomyłek i analiza błędów klasyfikacji	24
5.2. Wyniki modeli – zestawienie porównawcze	24
5.2.1. Wyniki modelu LSTM	24
5.2.2. Wyniki modelu Random Forest	24
5.2.3. Wyniki modelu XGBoost	24
5.3. Wzajemne porównanie modeli LSTM, Random Forest i XGBoost	24
5.4. Porównanie z klasycznymi metodami scoringowymi i istniejącymi rozwiązaniami	24
5.5. Interpretowalność modeli i ich przydatność decyzyjna	24
5.6. Dyskusja wyników i weryfikacja hipotez badawczych	24
Zakończenie	24
Bibliografia	24
Spis tabel	24
Spis rysunków i wykresów	24

Wstęp

Rozdział 1. Kredyt i ocena zdolności kredytowej – ujęcie teoretyczne

- 1.1. Pojęcie kredytu i ryzyka kredytowego
- 1.2. Tradycyjne metody oceny zdolności kredytowej
 - 1.2.1. Metody eksperckie i scoringowe
 - 1.2.2. Regulacje prawne i standardy branżowe
- 1.3. Ograniczenia klasycznych metod scoringowych
- 1.4. Kierunki modernizacji oceny zdolności kredytowej

Rozdział 2. Uczenie maszynowe w zastosowaniach finansowych

- 2.1. Podstawy uczenia maszynowego
 - 2.1.1. Taksonomia metod uczenia maszynowego
 - 2.1.2. Kluczowe pojęcia: cechy, etykiety, nadmierne dopasowanie, walidacja
- 2.2. Zastosowanie uczenia maszynowego w sektorze finansowym
 - 2.2.1. Przegląd zastosowań w zarządzaniu ryzykiem
 - 2.2.2. Credit scoring oparty na metodach ML – przegląd literatury
- 2.3. Charakterystyka wybranych algorytmów
 - 2.3.1. Sieci neuronowe z długą pamięcią krótkotrwałą (LSTM)
 - 2.3.2. Lasy losowe (Random Forest)
 - 2.3.3. Gradient Boosting – algorytm XGBoost

2.4. Porównanie algorytmów ML z klasycznymi metodami scoringowymi

2.5. Wyzwania etyczne i regulacyjne stosowania ML w ocenie kredytowej

Rozdział 3. Metodologia badań i projekt systemu

3.1. Cel i zakres badań

3.2. Postawione hipotezy badawcze

3.3. Struktura i charakterystyka danych dłużników

3.3.1. Źródła i sposób pozyskania danych

3.3.2. Opis zmiennych wejściowych i zmiennej docelowej

3.3.3. Preprocessing danych: czyszczenie, normalizacja, inżynieria cech

3.4. Projekt architektury systemu eksperymentalnego

3.4.1. Architektura ogólna i przepływ danych

3.4.2. Warstwa backendowa

3.4.3. Integracja modeli AI z systemem

3.4.4. Warstwa frontendowa

3.5. Obsługa wyjątków i scenariusze brzegowe

3.6. Narzędzia i środowisko technologiczne

Rozdział 4. Implementacja i trenowanie modeli

Tytuł niniejszej pracy akcentuje dynamiczne monitorowanie zdolności spłaty długu, a przyjęty w rozdziale 3 cel badawczy, wspomaganie oceny ryzyka kredytowego metodami uczenia maszynowego, znajduje w tym miejscu swoje bezpośrednie przełożenie na warstwę implementacyjną. W ramach pracy zaimplementowano trzy różne klasyfikatory reprezentujące odmienne rodziny algorytmiczne: Random Forest (zespół drzew z agregacją bagging), XGBoost (gradient boosting sekwencyjny) oraz LSTM (rekurencyjną sieć neuronową). Dwa pierwsze operują na statycznym, zagregowanym obrazie klienta i

odpowiadają klasycznemu ujęciu scoringowemu. Trzeci traktuje dane wejściowe jako szereg czasowy sześciu kolejnych miesięcy historii płatniczej, a więc modeluje progresję zachowań, a nie tylko ich jednorazowy stan. W tym ujęciu widoczne staje się znaczenie terminu „dynamiczne” z tytułu pracy: odnosi się on nie do cyklicznej aktualizacji modelu, lecz do architektury LSTM, która z założenia traktuje ocenę jako funkcję sekwencji zdarzeń, a nie pomiar statyczny.

Rozdział opisuje wszystkie decyzje projektowe związane z trenowaniem modeli: sposób podziału zbioru, strategię walidacji, techniki uwzględnienia niezbalansowania klas, dobór hiperparametrów, a także narzędzia analizy ważności cech (feature importance dla RF oraz wartości SHAP dla XGBoost). Całość kodu trenującego znajduje się w pliku *main.py*, do którego w dalszej części rozdziału odwołujemy się przy omawianiu konkretnych konstrukcji. Ocena jakości poszczególnych modeli, wraz z porównaniem metryk i interpretacją wyników biznesowych, zostanie przedstawiona w rozdziale 5.

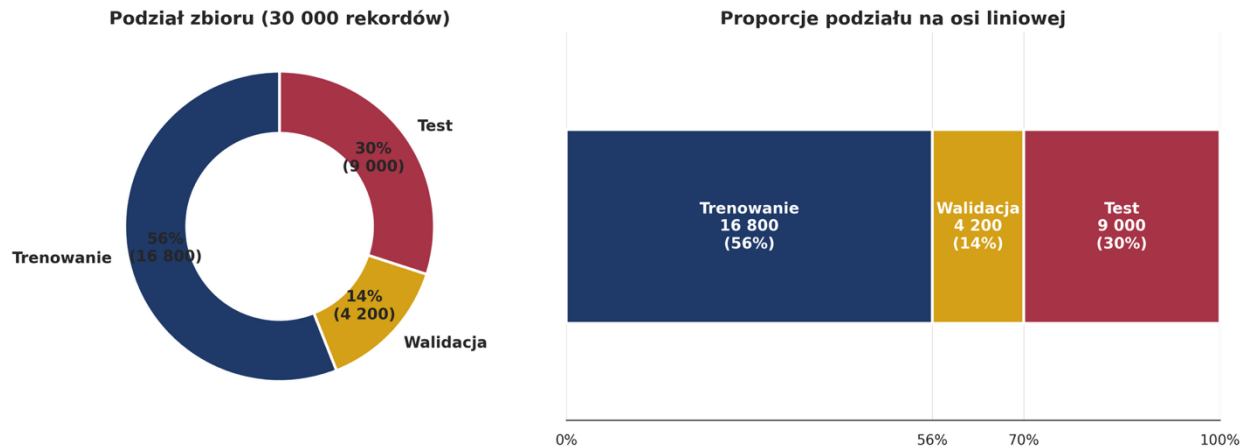
4.1. Podział zbioru danych i strategia walidacji

Rzetelna ocena modelu wymaga, by jego skuteczność była mierzona na danych, których w procesie trenowania nie oglądał. W przeciwnym razie raportowane metryki opisują jedynie zdolność do zapamiętywania, nie zaś do generalizacji. Przyjęta w pracy strategia opiera się na dwóch klasycznych rozwiązaniach: stratyfikowanym podziale na zbiór treningowy i testowy oraz, wyłącznie na potrzeby trenowania LSTM, dodatkowym wydzieleniu zbioru walidacyjnego z części treningowej.

4.1.1. Podział na zbiór treningowy, walidacyjny i testowy

Pełny zbiór UCI „default of credit card clients” liczy 30 000 rekordów. W pierwszym kroku wydzielono z niego zbiór testowy o udziale 30%, co odpowiada 9 000 obserwacji. Pozostałe 70% (21 000 rekordów) stanowi zbiór treningowy wspólny dla wszystkich trzech modeli. W przypadku LSTM z tego zbioru treningowego odcięto dodatkowo 20% jako zbiór walidacyjny, wykorzystywany przez mechanizm wczesnego zatrzymywania trenowania (EarlyStopping) oraz dynamiczną redukcję współczynnika uczenia (ReduceLROnPlateau). W efekcie otrzymano proporcje zilustrowane na rysunku 4.1: 56% trenowanie, 14% walidacja, 30% test.

Podział danych: train / val / test — stratyfikowany, seed = 42



Podział wykonywany jest z użyciem funkcji `train_test_split` z pakietu `scikit-learn`, z ustawionym parametrem `stratify=y`. Zachowuje to w każdym podzbiorze pierwotną proporcję klas, około 78% klientów spłacających i 22% niespłacających. Stratyfikacja ma tu istotne znaczenie praktyczne: przy losowym podziale bez stratyfikacji w teście mogłaby znaleźć się próbka z niższym udziałem klasy mniejszościowej, co sztucznie zawyżałoby dokładność i zniekształcało recall. Parametr `random_state=42` gwarantuje powtarzalność eksperymentu.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, stratify=y, random_state=42
)
```

Zbiór testowy pełni w eksperymencie rolę „zamrożoną”. Nie jest widziany przez modele na żadnym etapie trenowania ani doboru hiperparametrów. Dopiero w rozdziale 5, podczas finalnego raportu metryk, wchodzi on do analizy. Takie odseparowanie jest istotne, ponieważ nawet pozornie niewinne decyzje projektowe, na przykład wybór progu klasyfikacyjnego na podstawie wyników testowych, w praktyce prowadzą do wycieku informacji ze zbioru testowego do procesu trenowania, a tym samym do zawyżonej oceny modelu.

Zbiór walidacyjny LSTM wykorzystywany jest wyłącznie wewnętrznie, podczas trenowania sieci. Parametr `validation_split=0.2` przekazany do metody `fit` realizuje ten podział automatycznie, odcinając ostatnie 20% wierszy zbioru treningowego. Informacje stąd płynące, wartość straty walidacyjnej oraz metryka AUC na walidacji, służą wyłącznie

do sterowania procesem uczenia: zatrzymania go we właściwym momencie i obniżenia tempa uczenia, jeśli model utoyka na plateau. Rysunek 4.1 przedstawia ten sam podział w dwóch ujęciach jako wykres pierścieniowy oraz poziomy pasek proporcji, co pozwala jednocześnie odczytać udziały procentowe i bezwzględne liczebności podzbiorów.

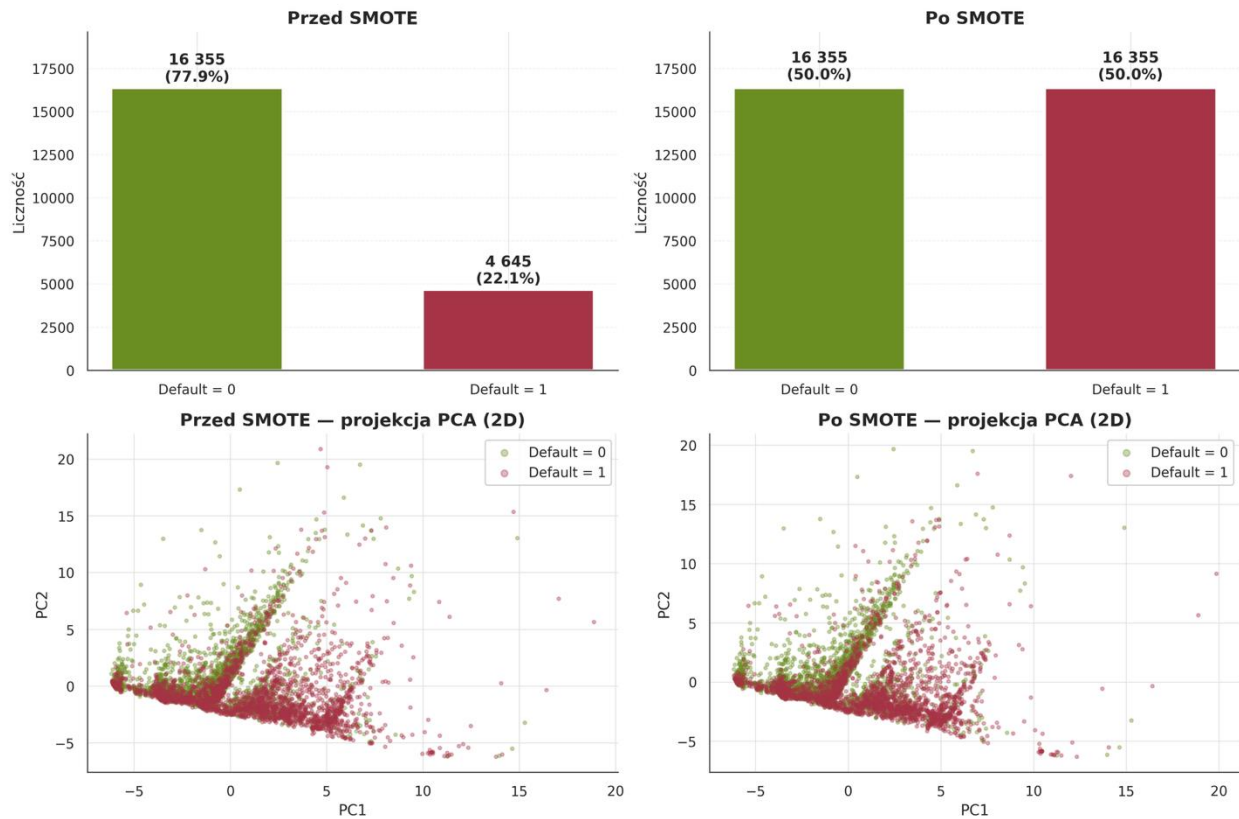
4.1.2. Techniki radzenia sobie z niezbalansowaniem klas (SMOTE, ważenie)

Zmienna docelowa *Default* w zbiorze UCI ma rozkład około 78/22 na korzyść klasy spłacającej. Niezbalansowanie to jest umiarkowane, w literaturze credit scoring bywają bowiem portfele, gdzie klasa mniejszościowa stanowi 2–3% rekordów. Niemniej nawet proporcja 22% wystarcza, by naiwne trenowanie na surowych danych prowadziło do modeli chętnie zwracających klasę większościową i bagatelizujących klasę klientów niespłacających. Z perspektywy celu pracy jest to sytuacja szczególnie niepożądana, ponieważ fałszywie negatywna diagnoza (przeoczenie ryzyka) kosztuje bank utratę kapitału, podczas gdy fałszywie pozytywna, jedynie utratę marży na niezawartej umowie. Priorytetem jest więc wysoki recall dla klasy mniejszościowej, nawet kosztem nieznacznego obniżenia precision.

W literaturze dwie rodziny metod dominują w podejściu do niezbalansowanych danych. Pierwsza modyfikuje rozkład zbioru treningowego poprzez oversampling klasy mniejszościowej lub undersampling klasy większościowej. Najbardziej rozpoznawalnym przedstawicielem tego nurtu jest metoda SMOTE (Synthetic Minority Over-sampling Technique), która generuje syntetyczne obserwacje klasy mniejszościowej przez interpolację liniową między najbliższymi sąsiadami w przestrzeni cech. Druga rodzina pozostawia dane nietknięte, a koryguje jedynie funkcję celu optymalizowaną przez klasyfikator, przypisując większą wagę pomyłkom na klasie mniejszościowej.

Rysunek 4.2 prezentuje efekt techniki SMOTE zastosowanej eksperymentalnie, na etapie analizy alternatyw, na pełnym zbiorze treningowym (21 000 obserwacji wykorzystywanym przez Random Forest i XGBoost); w finalnym pipeline zrezygnowano z tej metody na rzecz ważenia klas opisanego poniżej. Wykres przedstawia efekt SMOTE w dwóch ujęciach. Pierwszy wiersz pokazuje liczebność klas przed i po, po zastosowaniu SMOTE każda klasa liczy około 16 tysięcy obserwacji. Drugi wiersz zawiera rzut PCA na przestrzeń dwuwymiarową, co pozwala ocenić, czy syntetyczne próbki rozkładają się sensownie względem oryginalnych. Obserwujemy, że nowe punkty klasy mniejszościowej zagęszczają obszary zajmowane już wcześniej przez prawdziwe przykłady i nie tworzą struktur oderwanych od danych oryginalnych. Samo w sobie nie jest to jednak gwarancja bezpieczeństwa, interpolacja liniowa w wielowymiarowej przestrzeni finansowej bywa ryzykowna, bo może łączyć przypadki powierzchownie podobne, lecz merytorycznie różne (np. klienci o zbliżonym limicie, lecz zupełnie innym wzorcu zadłużenia).

Wpływ techniki SMOTE na balans klas w zbiorze treningowym



Z tego powodu w finalnej implementacji zdecydowano się na drugie podejście. Dla Random Forest oraz LSTM wykorzystano mechanizm `class_weight="balanced"`, który w trakcie trenowania waży obserwacje odwrotnie proporcjonalnie do częstości klasy. W przypadku LSTM wagi obliczane są jawnie funkcją `compute_class_weight` i przekazywane do metody `fit` jako słownik:

```
lstm_class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.array([0, 1]),
    y=y_train_seq.values
)
lstm_class_weight_dict = {0: lstm_class_weights[0], 1: lstm_class_weights[1]}
```

Dla XGBoost równoważnym mechanizmem jest parametr `scale_pos_weight`, wyliczany jako stosunek liczebności klasy większościowej do mniejszościowej:

```
xgb = XGBClassifier(
    ...
```

```
scale_pos_weight=(len(y) - sum(y)) / sum(y),  
...  
)
```

Powodów, dla których zrezygnowano z SMOTE na rzecz ważenia klas, jest kilka. Po pierwsze, ważenie nie zmienia rozkładu danych, oryginalne obserwacje pozostają nietknięte, a model uczy się ich naturalnej struktury, jedynie przywiązując większą wagę do błędów na klasie mniejszościowej. Po drugie, SMOTE wprowadza syntetyczne dane, które, choć lokalnie zgodne z rozkładem oryginału, nie odzwierciedlają rzeczywistej populacji klientów; w kontekście bankowym takie przekłamanie jest trudne do uzasadnienia przed regulatorem. Po trzecie, *class_weight* i *scale_pos_weight* są rozwiązaniami zintegrowanymi z biblioteką, co eliminuje dodatkowy etap preprocessingu i zmniejsza powierzchnię błędu w pipeline.

4.2. Implementacja modelu LSTM

Model LSTM (Long Short-Term Memory) jest w pracy głównym nośnikiem dynamicznego ujęcia sformułowanego w tytule. W odróżnieniu od klasyfikatorów z sekcji 4.3 i 4.4, które operują na wektorze cech agregujących historię klienta do pojedynczej liczby lub zestawu liczb, LSTM otrzymuje na wejściu uporządkowaną sekwencję sześciu miesięcy historii płatniczej i uczy się rozpoznawać wzorce rozwijające się w czasie. Z perspektywy oceny zdolności spłaty oznacza to, że model nie zadowala się odpowiedzią na pytanie „jaki jest klient dzisiaj?”, lecz próbuje uchwycić trajektorię prowadzącą do tego stanu.

4.2.1. Dobór architektury sieci i hiperparametrów

Architekturę sieci dobrano z zamiarem utrzymania równowagi między zdolnością uogólniania a ryzykiem nadmiernego dopasowania (overfitting). Wejście sieci ma wymiar (6, 3): sześć kroków czasowych reprezentujących kolejne miesiące oraz trzy cechy na każdy krok, status płatności (*PAY*), saldo rachunku (*BILL_AMT*) i kwota wpłaty (*PAY_AMT*). Po warstwie wejściowej znajduje się pojedyncza warstwa LSTM o 32 jednostkach ukrytych, warstwa Dropout z prawdopodobieństwem 0,3, w pełni połączona warstwa Dense(16) z funkcją aktywacji ReLU oraz pojedynczy neuron wyjściowy z funkcją sigmoid, zwracający szacowane prawdopodobieństwo niewypłacalności. Całą konstrukcję opisuje następujący fragment kodu:

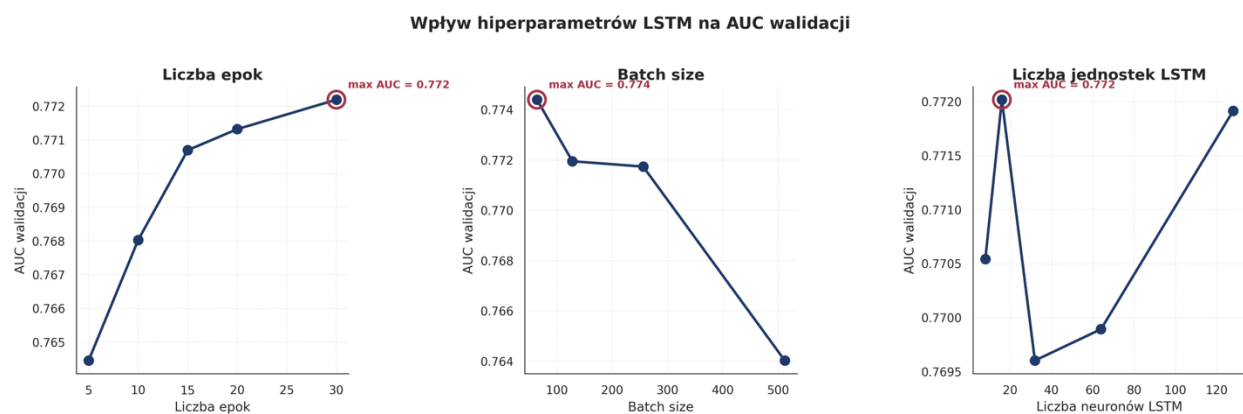
```
model = Sequential([  
    Input(shape=(6, 3)),  
    LSTM(32),  
    Dropout(0.3),
```

```

Dense(16, activation="relu"),
Dense(1, activation="sigmoid")
})

```

Każdy z tych wyborów ma uzasadnienie empiryczne. Liczba 32 jednostek LSTM wynika z analizy przedstawionej na rysunku 4.4, na którym w jednym z paneli porównano AUC walidacyjne dla warstw o 16, 32, 64 i 128 jednostkach. Mniejsza liczba jednostek (16) skutkuje niedouczeniem, sieć nie jest w stanie zakodować subtelnych zależności w sekwencji. Większe warstwy (64, 128) osiągają AUC walidacyjne praktycznie identyczne z wariantem 32-jednostkowym, lecz kosztem wyraźnie wyższej liczby parametrów, dłuższego trenowania oraz większego ryzyka przeuczenia na dłuższych przebiegach. Wybór 32 jednostek uzasadniony jest zasadą oszczędności modelu. Przy tej samej skuteczności predykcyjnej preferowana jest architektura prostsza, lepiej regularyzowana przez Dropout i tańsza w eksploatacji.



Dropout z prawdopodobieństwem 0,3 pełni rolę regularyzatora, wymuszając rozproszenie informacji w obrębie warstwy i ograniczając zależność predykcji od pojedynczych neuronów (szczegóły teoretyczne w sekcji 2.3.1). Wartości poniżej 0,2 dawały zbyt słabą regularyzację i sieć wykazywała oznaki przeuczenia, a powyżej 0,4 prowadziły do niedouczenia. Model tracił wówczas zdolność stabilnego uczenia wzorców. Warstwa Dense(16) pełni rolę klasyfikatora pracującego na wektorze cech ukrytych, zwróconym przez LSTM, a sigmoidalna warstwa wyjściowa mapuje wynik na przedział [0, 1] interpretowalny jako prawdopodobieństwo klasy dodatniej.

Osobnego komentarza wymaga sposób, w jaki dane zorganizowano w wymiar czasowy. W oryginalnym zbiorze UCI kolumny *PAY_0*, *PAY_2*, *PAY_3*, *PAY_4*, *PAY_5*, *PAY_6* (przyjęta w źródle numeracja pomija *PAY_1*) reprezentują status płatności odpowiednio za wrzesień, sierpień, lipiec, czerwiec, maj i kwiecień. W implementacji kolejność kolumn

została odwrócona, w tablicy wejściowej pierwszy krok czasowy odpowiada kwietniowi, a ostatni wrześniowi:

```
pay_seq_cols = ["PAY_6", "PAY_5", "PAY_4", "PAY_3", "PAY_2", "PAY_0"]
bill_seq_cols = ["BILL_AMT6", "BILL_AMT5", "BILL_AMT4", "BILL_AMT3", "BILL_AMT2", "BILL_AMT1"]
pay_amt_seq_cols = ["PAY_AMT6", "PAY_AMT5", "PAY_AMT4", "PAY_AMT3", "PAY_AMT2", "PAY_AMT1"]
```

Taki porządek ma istotne znaczenie merytoryczne. LSTM, przetwarzając sekwencję w porządku chronologicznym (od najstarszego do najnowszego miesiąca), aktualizuje swoje stany ukryte w kierunku przepływu czasu i uczy się progresji zachowań klienta, a nie ich statycznego obrazu. Najnowsze obserwacje, z wrześniowego salda i statusu płatności, trafiają do sieci na końcu sekwencji i mają największy wpływ na ostatni stan ukryty, z którego warstwa Dense generuje predykcję. Gdyby kolejność była odwrotna, sieć uczyłaby się tych samych zależności statystycznie, ale w kierunku wstecznym, co utrudnia interpretację modelu i kłóci się z intuicją biznesową: w praktyce bankowej analityk także rozpatruje historię od najstarszego wpisu do najnowszego.

Każda z trzech cech (PAY, BILL_AMT, PAY_AMT) została skalowana osobno, z użyciem niezależnego obiektu *StandardScaler*. Uzasadnieniem takiego wyboru jest wyraźnie różna skala wartości poszczególnych cech: *PAY* przyjmuje małe wartości całkowite, najczęściej w przedziale -2 do 8, *BILL_AMT* sięga setek tysięcy, a *PAY_AMT* również operuje w wysokich przedziałach, ale z zupełnie innym rozkładem niż *BILL_AMT*. Wspólne skalowanie zniekształciłoby zależności wewnątrz każdej cechy. Obiekty skalujące są zachowywane i wykorzystywane również w serwisie predykcyjnym (pliku *lstm_scalers.pkl*), co zapewnia spójność transformacji pomiędzy etapem trenowania a etapem wnioskowania.

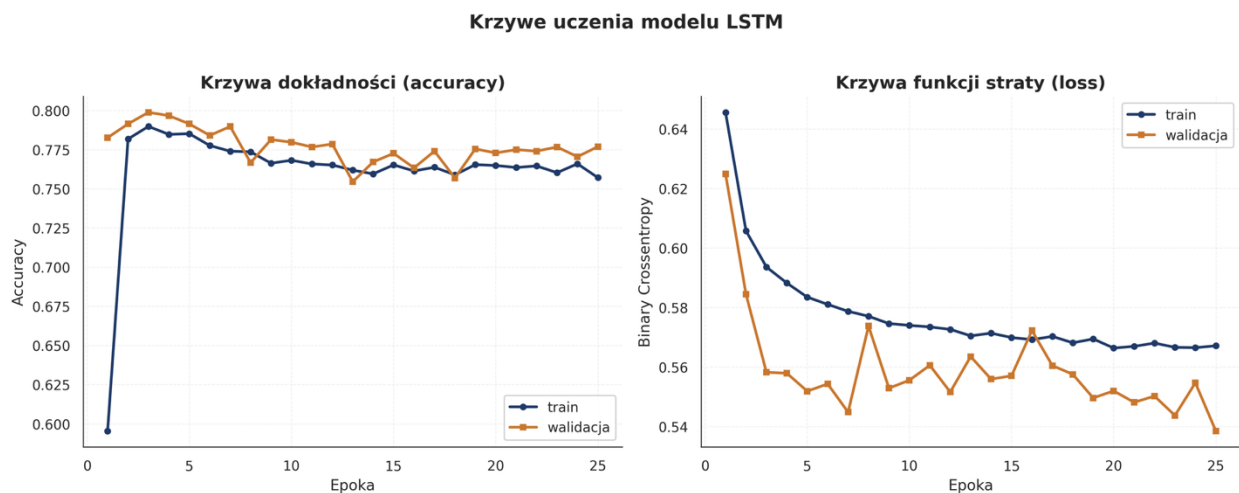
4.2.2. Proces trenowania i krzywe uczenia

Sieć trenowana jest algorytmem Adam z domyślnymi ustawieniami, funkcją straty *binary_crossentropy* oraz dwiema metrykami: ogólnym AUC i jawnie, poprzez callback, AUC walidacyjnym sterującym mechanizmem *EarlyStopping*. Trenowanie odbywa się przy batch size równym 256 oraz maksymalnej liczbie epok ustalonej na 60. W praktyce pełne 60 epok jest rzadko osiąganę, ponieważ *EarlyStopping* zatrzymuje proces, gdy przez pięć kolejnych epok metryka walidacyjna przestaje się poprawiać, a następnie przywraca wagi modelu z momentu najlepszego wyniku.

```
lstm_callbacks = [
    EarlyStopping(monitor="val_auc", mode="max", patience=5, restore_best_weights=True),
    ReduceLROnPlateau(monitor="val_auc", mode="max", factor=0.5, patience=3, min_lr=1e-5),
]
```

Drugi z callbacków ReduceLROnPlateau działa jako subtelniejsza forma adaptacji. Jeśli przez trzy epoki z rzędu AUC walidacyjne nie rośnie, współczynnik uczenia jest zmniejszany o połowę, aż do minimum 10^{-5} . Pozwala to sieci wyjść z płytkich minimów lokalnych, które czasem pojawiają się w okolicach plateau, bez konieczności sztywnego ustawiania harmonogramu uczenia (learning rate schedule) z góry.

Na rysunku 4.3 przedstawiono krzywe uczenia z typowego przebiegu treningu. Panel lewy pokazuje dokładność (accuracy), a panel prawy wartość funkcji straty; w obu przypadkach dla zbioru treningowego i walidacyjnego. Można zauważyć, że accuracy rośnie szybko w pierwszych epokach, a następnie stabilizuje się na poziomie zbliżonym do wartości testowej. Istotniejsza dla oceny zdrowia procesu jest strata walidacyjna. Jej monotonicznie malejący przebieg przez większość epok świadczy o braku przeuczenia. Charakterystyczny symptom overfittingu, rosnąca strata walidacyjna przy nadal malejącej stracie treningowej, nie występuje, co jest zasługą dwóch mechanizmów działających łącznie: warstwy Dropout i wczesnego zatrzymywania.



Rysunek 4.4 uzupełnia ten obraz, pokazując, jak na AUC walidacyjne wpływają trzy hiperparametry: liczba epok, wielkość batcha oraz liczba jednostek LSTM. Każdy panel zawiera czerwone oznaczenie optimum. Analizując przebiegi, można zauważyć klasyczne „kolanko”: po ~15 epokach AUC osiąga plateau, a kolejne iteracje jedynie nieznacznie modyfikują wartości metryki; batch size poniżej 128 wprowadza nadmierny szum w gradiencie i spowalnia konwergencję, a powyżej 512, choć szybszy obliczeniowo, daje nieco słabszą stabilność numeryczną; liczba jednostek 32 okazuje się optymalna w sensie AUC, co potwierdza wybór dokonany na etapie projektowania.

LSTM w zaprojektowanej architekturze charakteryzuje się niskim kosztem obliczeniowym. Pełny trening na procesorze trwa kilka minut, co pozwala swobodnie eksperymentować z hiperparametrami bez konieczności wykorzystania GPU. Jest to konsekwencja krótkiego horyzontu czasowego (zaledwie 6 kroków) i niewielkiej liczby cech, sytuacja typowa dla scoringu kredytowego, odmienna od problemów NLP, gdzie sekwencje liczą setki lub tysiące tokenów.

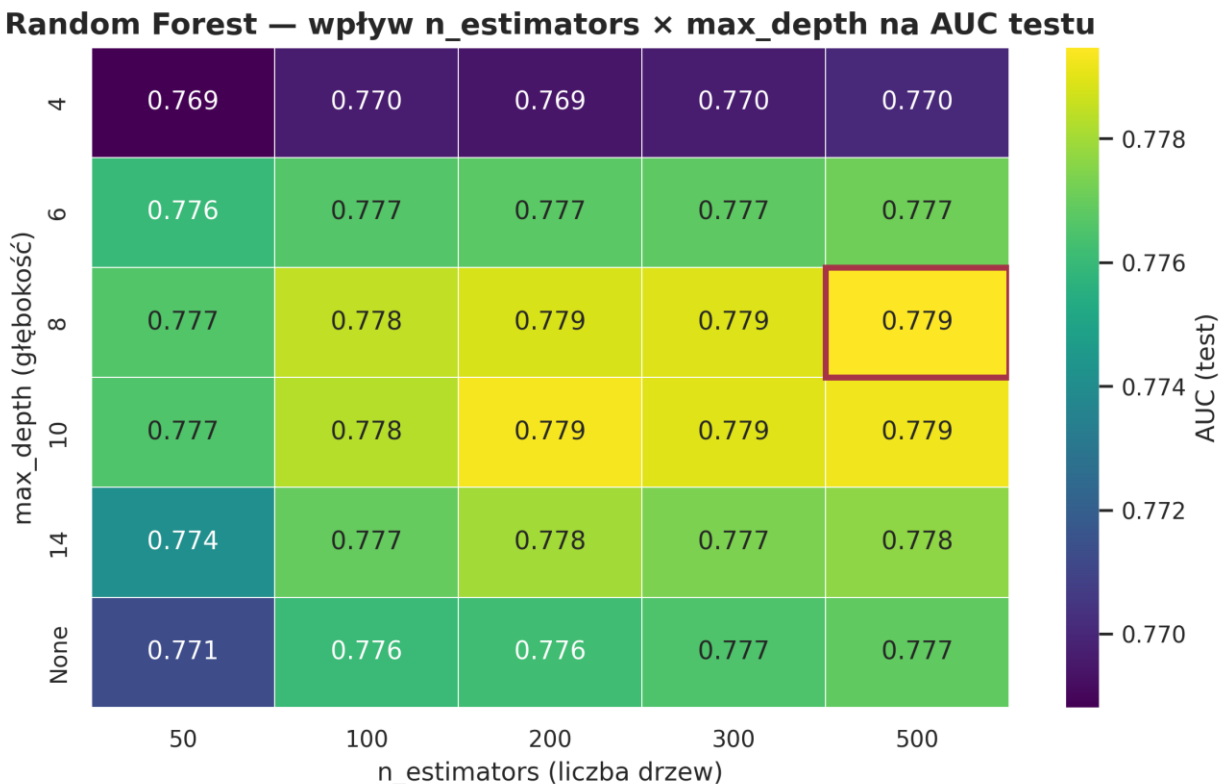
4.3. Implementacja modelu Random Forest

Random Forest jest w pracy reprezentantem rodziny ensemble bagging, zespołu drzew decyzyjnych trenowanych niezależnie na bootstrapowych próbkach danych, których predykcje są agregowane przez głosowanie większościowe. Z punktu widzenia praktyki bankowej zaletą tego podejścia jest relatywnie dobra interpretowalność poprzez ważność cech (feature importance), szybkość trenowania oraz stabilność wyników, pojedyncze drzewo jest słabym klasyfikatorem, lecz zespół redukuje wariancję i minimalizuje wpływ przypadkowych podziałów.

4.3.1. Dobór liczby drzew i głębokości

Konfiguracja klasyfikatora została ustalona na podstawie siatki wartości przetestowanych wzdłuż dwóch najistotniejszych hiperparametrów: liczby drzew (*n_estimators*) i maksymalnej głębokości (*max_depth*). Wyniki tego eksperymentu prezentuje rysunek 4.5, na którym w postaci heatmapy zestawiono AUC testowe dla poszczególnych kombinacji. Czerwone obramowanie oznacza konfigurację ostatecznie przyjętą:

```
rf = RandomForestClassifier(  
    n_estimators=500,  
    max_depth=10,  
    min_samples_leaf=5,  
    class_weight="balanced",  
    random_state=42  
)
```



Liczbę drzew ustalono na 500, ponieważ dalsze jej zwiększanie powodowało jedynie marginalny przyrost AUC przy kwadratowym wzroście kosztu obliczeniowego, różnica między 500 a 1000 drzewami była na poziomie trzeciego miejsca po przecinku, niewspółmierna do dwukrotnego wydłużenia czasu trenowania. Jednocześnie 500 jest wartością na tyle dużą, by agregacja predykcji skutecznie redukowałą wariancję, co potwierdza widoczna w górnej części heatmapy asymptotyczność krzywej AUC względem liczby drzew.

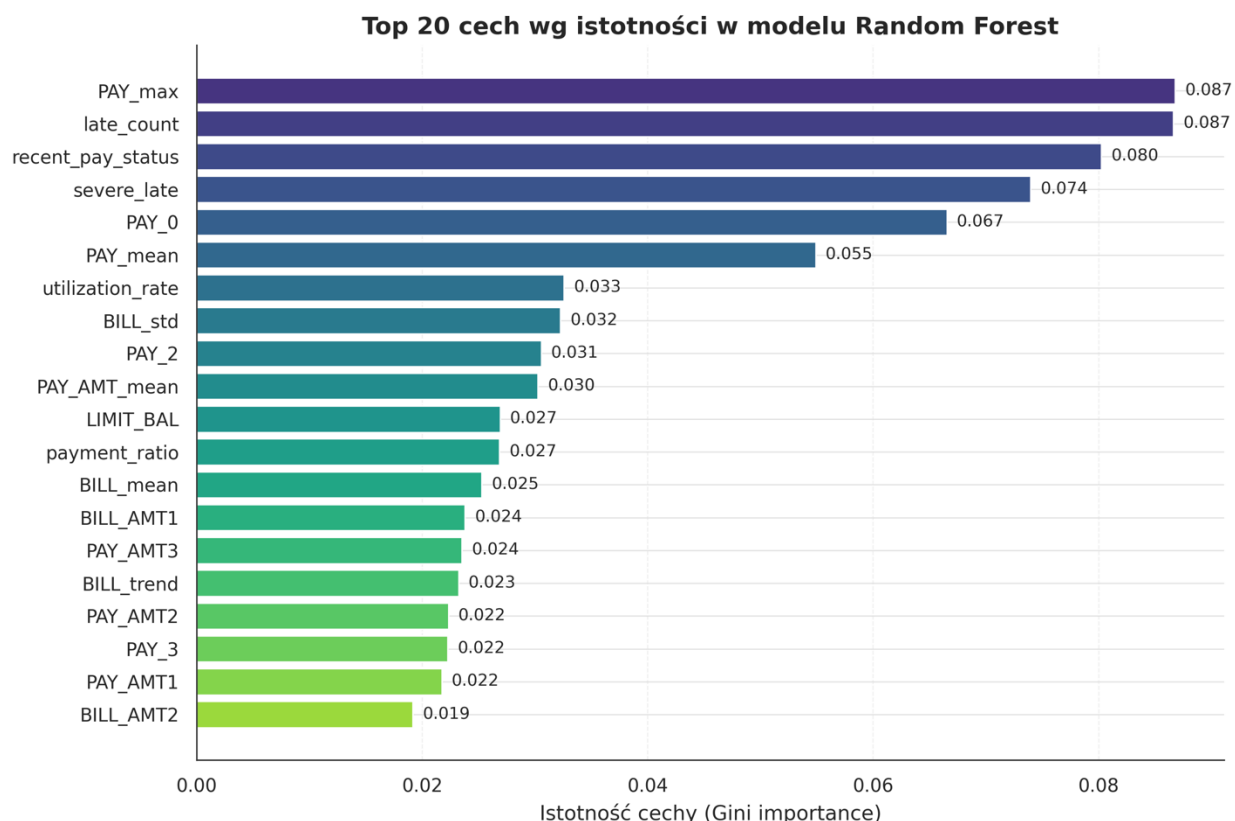
Wybór $max_depth=10$ był mniej oczywisty. Drzewa płytsze (3–6) okazywały się niedouczone, zbyt słabo rozpoznawały nieliniowe zależności w cechach takich jak *utilization_rate* czy *BILL_trend*. Drzewa głębokie lub nieograniczone ($max_depth=None$) osiągały bardzo wysokie AUC treningowe, lecz na zbiorze testowym ich wyniki zaczynały spadać, klasyczny symptom overfittingu. Głębokość 10 zachowuje balans między tymi skrajnościami, pozwala na uchwycenie interakcji drugiego i trzeciego rzędu między cechami, lecz nie dopuszcza do zapamiętywania szumów pojedynczych obserwacji. Na heatmapie z rysunku 4.5 konfiguracje $max_depth=8$ i $max_depth=10$ dają praktycznie identyczne AUC (0,779); wybór głębszej wartości uzasadniony jest stabilnością rankingu cech przy tej samej wielkości lasu.

Pomocniczo ustawiono również parametr *min_samples_leaf=5*, który wymusza, by w każdym liściu drzewa znajdowało się co najmniej pięć obserwacji. Jest to dodatkowa bariera przeciwko nadmiernemu dopasowaniu, zapobiega tworzeniu liści reprezentujących pojedyncze rekordy, a zatem wzmacnia stabilność predykcji. Parametr *class_weight="balanced"* odpowiada za opisaną w sekcji 4.1.2 kompensację niezbalansowania, a *random_state=42* zapewnia pełną powtarzalność wyników.

4.3.2. Ważność cech

Istotną zaletą Random Forest jest łatwość oszacowania wpływu poszczególnych cech na predykcję. Atrybut *feature_importances_* modelu zwraca znormalizowane wartości oparte na średnim zmniejszeniu nieczystości Gini w węzłach podziału, agregowane po wszystkich drzewach zespołu. Jakkolwiek miara ta jest krytykowana za zawyżanie istotności cech numerycznych o wysokiej kardynalności, w zestawieniu z dobrze dobranymi cechami sekwencyjnymi i inżynierskimi daje obraz spójny z oczekiwaniami analityków kredytowych.

Na rysunku 4.6 przedstawiono dwadzieścia najistotniejszych cech, posortowanych malejąco. Na szczycie listy znalazły się zmienne związane z najnowszym i najbardziej dotkliwym zachowaniem płatniczym: *PAY_max*, *late_count* oraz *recent_pay_status*. Pierwsza, *PAY_max*, reprezentuje najgorszy status płatności w sześciomiesięcznym oknie. Wychwytuje zatem pojedyncze, skrajne epizody opóźnień, które nie byłyby widoczne w uśrednionych miarach. Druga, *late_count*, zlicza, w ilu z sześciu miesięcy klient był w jakimkolwiek opóźnieniu. Trzecia, *recent_pay_status*, odzwierciedla status płatności w ostatnim dostępnym miesiącu; w implementacji stanowi ona kopię *PAY_0* i plasuje się na liście istotności nieco niżej niż sam *PAY_0*, co jest naturalnym efektem konstrukcji Random Forest — losowy wybór podzbioru cech w węzłach powoduje „rozmywanie” istotności między cechami silnie skorelowanymi, a nie świadczy o niedopatrzeniu implementacyjnym. Dominacja tych trzech zmiennych potwierdza wnioski znane z literatury credit scoring: najlepszym predyktorem przyszłego niewywiązywania się ze zobowiązań jest świeża historia takich zdarzeń. Dla praktyki bankowej ma to wymierne znaczenie, sugeruje, że systemy wczesnego ostrzegania powinny w pierwszej kolejności reagować na zmianę statusu płatności w bieżącym cyklu rozliczeniowym.



W dalszej części rankingu pojawiają się cechy inżynierskie utworzone w rozdziale 3: *utilization_rate* (stosunek średniego salda do limitu), *BILL_trend* (zmiana salda między kwietniem a wrześniem), *severe_late* (flaga poważnego opóźnienia) oraz *payment_ratio* (relacja wpłat do rachunków). Fakt, że cechy te plasują się wyżej niż niektóre zmienne surowe z pliku źródłowego, stanowi liczbowy dowód wartości feature engineeringu, nie jest to jedynie deklaracja, lecz efekt widoczny w istotności atrybutów wyuczonego modelu.

W dolnej części rankingu znajdują się zmienne demograficzne: *AGE*, *EDUCATION*, *MARRIAGE* oraz *SEX*. Ich niska istotność jest obserwacją o dwóch konsekwencjach. Po pierwsze, metodologicznej, w tym zbiorze zachowanie płatnicze jest zdecydowanie silniejszym predyktorem niż profil demograficzny. Po drugie, etycznej i regulacyjnej, ograniczona waga zmiennych chronionych, takich jak płeć, zmniejsza ryzyko dyskryminacji pośredniej w modelu, co jest istotnym argumentem przed oceną zgodności z regulacjami (przede wszystkim z unijnym AI Act i dotychczasowym RODO).

4.4. Implementacja modelu XGBoost

XGBoost (eXtreme Gradient Boosting) buduje zespół drzew sekwencyjnie — każde kolejne drzewo koryguje błędy poprzedników — w odróżnieniu od niezależnego trenowania

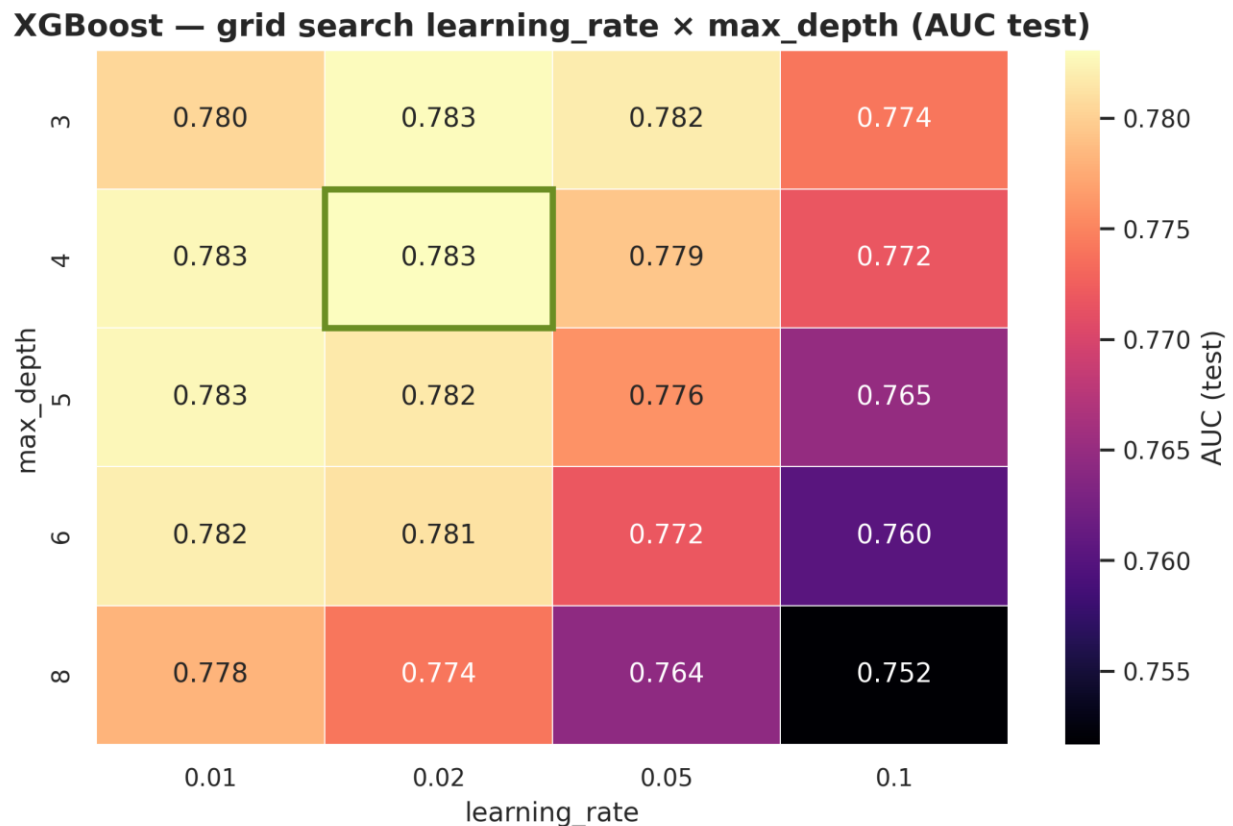
z agregacją głosowaniem stosowanego w Random Forest (szczegóły teoretyczne w sekcji 2.3.3). Efektem takiego podejścia jest zazwyczaj wyższa skuteczność predykcyjna, lecz także większa wrażliwość na hiperparametry oraz wyższe ryzyko overfittingu, wymagające staranniejszej regularyzacji. W pracy XGBoost pełni rolę modelu referencyjnego dla statycznego ujęcia problemu, to na nim najczęściej porównuje się nowe metody w benchmarkach credit scoring.

4.4.1. Strojenie hiperparametrów – Grid Search i Cross-Validation

Konfiguracja XGBoost w finalnej implementacji wygląda następująco:

```
xgb = XGBClassifier(  
    n_estimators=800,  
    learning_rate=0.02,  
    max_depth=4,  
    subsample=0.7,  
    colsample_bytree=0.7,  
    reg_alpha=0.1,  
    reg_lambda=1.0,  
    scale_pos_weight=(len(y) - sum(y)) / sum(y),  
    random_state=42,  
    eval_metric='auc'  
)
```

Dobór kluczowej pary parametrów, *learning_rate* oraz *max_depth*, przeprowadzono w oparciu o siatkę wartości (grid search). Wyniki zaprezentowano na rysunku 4.7, również w postaci heatmapy AUC testowego. Zielone obramowanie wskazuje konfigurację optymalną, która okazała się spójna z zaleceniami twórców XGBoost: niski współczynnik uczenia (0,01–0,02) w połączeniu z niewielką głębokością drzew (3–4). W algorytmie boostingu każde drzewo wnosi niewielką korektę, a duża liczba iteracji pozwala ją stopniowo akumulować. Wysokie *learning_rate* przy dużej głębokości prowadzi do szybkiego przeuczenia, co w heatmapie widać jako ciemne pola w prawym dolnym narożniku.



Niskie $\text{learning_rate}=0.02$ wymaga rekompensaty w postaci dużej liczby iteracji. Wybrana wartość 800 okazała się wystarczająca, kolejne eksperymenty z $n_estimators=1200$ dawały już tylko znikomą poprawę AUC kosztem około dwukrotnie dłuższego trenowania. Dodatkowo zastosowano dwie techniki regularyzacji: stochastyczne próbkowanie zarówno obserwacji ($\text{subsample}=0.7$), jak i cech ($\text{colsample_bytree}=0.7$), oraz klasyczną regularyzację L1 i L2 ($\text{reg_alpha}=0.1$, $\text{reg_lambda}=1.0$). Takie połączenie redukuje wariancję modelu przy zachowaniu wysokiej skuteczności predykcyjnej.

Walidacja krzyżowa (ang. cross-validation) zaprezentowana na rysunku 4.9 stanowi standardową technikę oceny stabilności modelu, w każdej z pięciu iteracji inny fold pełni rolę zbioru walidacyjnego, a reszta służy do trenowania. Takie rozwiązanie pozwala oszacować wariancję metryki pomiędzy próbami, dając pojęcie o wrażliwości modelu na konkretny podział danych. W niniejszej pracy walidację krzyżową 5-fold zaprezentowano schematycznie, natomiast do właściwej oceny modeli przyjęto pojedynczy, stratyfikowany podział treningowo-testowy 70/30 opisany w sekcji 4.1.1. Powodem tej decyzji był koszt obliczeniowy: pełna walidacja krzyżowa wymagałaby pięciokrotnego trenowania modeli z 500 drzewami (Random Forest) i 800 iteracjami boostingu (XGBoost), co w połączeniu z eksperymentami dla kilkunastu kombinacji hiperparametrów, znacznie wydłużałoby czas

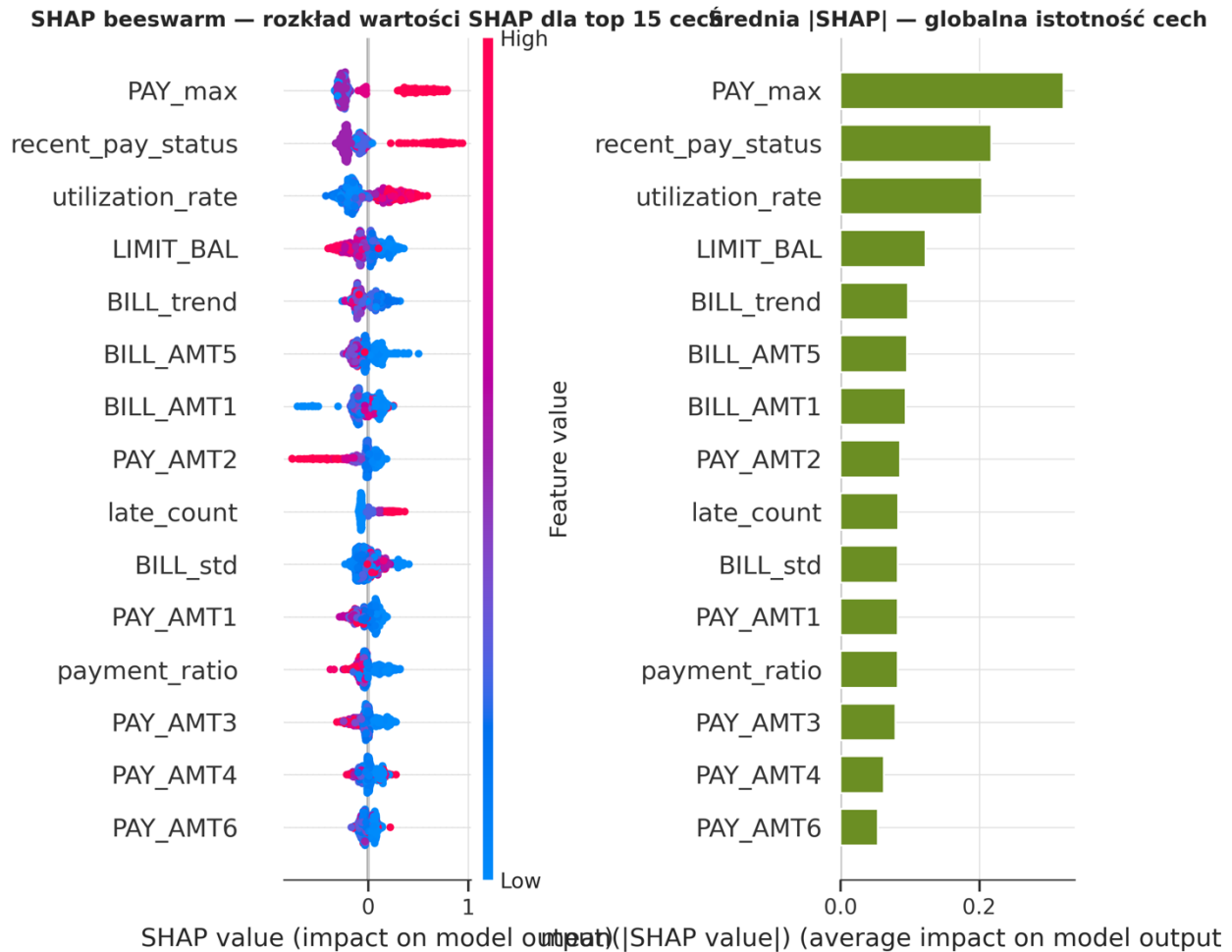
pojedynczego cyklu badawczego. Stratyfikowany podział 70/30 z *random_state=42* daje stabilną, powtarzalną ocenę modelu, a wyniki na niezależnym zbiorze testowym nie wykazują oznak niestabilności, które uzasadniałyby koszty pełnego CV.

4.4.2. Analiza wkładu cech (SHAP values)

Jednym z głównych zarzutów stawianych modelom opartym na drzewach boostingu jest ich zredukowana interpretowalność w porównaniu z prostymi metodami scoringowymi. O ile w pojedynczym drzewie decyzyjnym da się odtworzyć logikę decyzji, o tyle w zespole ośmiuset drzew taka inspekcja byłaby bezużyteczna. Odpowiedzią na ten problem jest metoda SHAP (Shapley Additive exPlanations), której podstawą teoretyczną jest klasyczna teoria gier kooperacyjnych Shapleya. Każda cecha modelu traktowana jest jako „gracz”, a jej wkład do predykcji, jako odpowiednik wartości Shapleya w grze, tj. średniego wkładu tego gracza do wyniku końcowego, wyznaczonego po wszystkich możliwych koalicjach. Jest to jedyne sformułowanie spełniające jednocześnie trzy aksjomaty zaproponowane przez Lundberga i Lee (2017): lokalną dokładność (local accuracy), własność braku (missingness) oraz konsekwencję (consistency).

Rysunek 4.8 przedstawia dwie komplementarne wizualizacje wartości SHAP. Lewa część zawiera tak zwany beeswarm plot. Każda kropka odpowiada pojedynczej obserwacji zbioru testowego, a jej pozioma pozycja, wartości SHAP dla danej cechy przy tej obserwacji. Kolor kropki koduje wartość cechy (ciemnoczerwony, wartości wysokie, niebieski niskie). Dzięki temu jeden wykres daje wgląd w cztery aspekty jednocześnie: kierunek wpływu cechy, jego rozpiętość, zależność od wartości cechy oraz rozkład populacji klientów.

Wyjaśnialność modelu XGBoost — wartości SHAP (n=1000)



Prawa część rysunku zawiera bar plot, zestawienie średniej wartości bezwzględnej SHAP dla każdej cechy. Jest to uśredniona, globalna istotność cech, analogiczna do ważności cech z sekcji 4.3.2, lecz oparta na innym fundamencie teoretycznym. Porównując obie wizualizacje z listą istotności Random Forest, można zauważyć wysoką zgodność uporządkowania: status płatności z ostatniego miesiąca, zbiorcza miara opóźnień oraz wskaźnik wykorzystania limitu konsekwentnie zajmują czołowe pozycje, niezależnie od algorytmu. Zgodność rankingów wzmacnia wiarygodność wniosków. Ranking cech nie jest efektem wybranej rodziny algorytmicznej, lecz odzwierciedla strukturę zależności w danych.

Dla praktyki bankowej istotne są szczególnie wyjaśnienia lokalne. SHAP pozwala wyjaśnić każdą pojedynczą predykcję modelu w postaci addytywnej: predykcja = wartość

bazowa + suma wkładów cech. Dla analityka kredytowego oznacza to możliwość udzielenia klientowi precyzyjnej odpowiedzi, dlatego jego wniosek został sklasyfikowany jako ryzykowny, z wymienieniem konkretnych cech, które przeważały decyzję. W kontekście wymagań unijnego AI Act oraz postanowień RODO dotyczących zautomatyzowanego podejmowania decyzji (art. 22), zdolność do generowania wyjaśnień lokalnych na poziomie pojedynczej predykcji stanowi istotną przewagę XGBoost jako kandydata na model produkcyjny.

4.5. Walidacja i testowanie modeli w środowisku eksperymentalnym

Po zakończeniu trenowania każdy z trzech modeli jest ewaluowany na zamrożonym zbiorze testowym, tym samym dla Random Forest i XGBoost, a dla LSTM na jego odpowiedniku z tablicy trójwymiarowej `X_test_seq`. Główną metryką raportowaną w pętli trenującej jest ROC-AUC, obliczana funkcją `roc_auc_score` z pakietu `scikit-learn`:

```
y_pred_rf = rf.predict_proba(X_test)[:, 1]
y_pred_xgb = xgb.predict_proba(X_test)[:, 1]
y_pred_lstm = model.predict(X_test_seq).ravel()
```

Wybór ROC-AUC jako metryki wiodącej nie jest przypadkowy. Przy niezbalansowanym rozkładzie klas (78/22) accuracy bywa myląca, nawet klasyfikator zwracający zawsze klasę większościową osiąga 78% dokładności, choć z perspektywy biznesowej jest całkowicie bezużyteczny. ROC-AUC jest natomiast niezależna od wyboru progu klasyfikacyjnego oraz od proporcji klas, co czyni ją standardem w credit scoring. W kolejnym rozdziale zostanie ona uzupełniona o metryki progowe: precision, recall, F1, których omówienie pozwoli głębiej zinterpretować różnice między modelami z perspektywy kosztów błędnych decyzji kredytowych.

Zastosowano pojedynczy, stratyfikowany podział 70/30 opisany w sekcji 4.1.1; uzasadnienie rezygnacji z pełnej walidacji krzyżowej zostało omówione w sekcji 4.4.1. Rysunek 4.9 prezentuje schematycznie technikę 5-fold CV jako punkt odniesienia: w każdej iteracji inny fold pełni rolę walidacji, a pozostałe cztery służą trenowaniu. Wariant stratyfikowany zachowuje w każdym foldzie proporcję klas 78/22, co przy niezbalansowanych danych jest wymogiem koniecznym. Stabilność wyników potwierdza dodatkowo raportowana w rozdziale 5 wariancja AUC, obliczona na podstawie 40 bootstrapowanych powtórzeń zbioru testowego. Jeśli model jest stabilny w pojedynczym splicie, przejście do pełnego CV zmienia jedynie przedział ufności, a nie ranking metod.

Walidacja krzyżowa 5-fold (stratified) — rotacja zbioru walidacyjnego

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Iteracja 1	walidacja	trenowanie	trenowanie	trenowanie	trenowanie
Iteracja 2	trenowanie	walidacja	trenowanie	trenowanie	trenowanie
Iteracja 3	trenowanie	trenowanie	walidacja	trenowanie	trenowanie
Iteracja 4	trenowanie	trenowanie	trenowanie	walidacja	trenowanie
Iteracja 5	trenowanie	trenowanie	trenowanie	trenowanie	walidacja

Zbiór treningowy Zbiór walidacyjny

Każdy z wytrenowanych modeli jest na końcu serializowany do pliku: Random Forest i XGBoost za pomocą *joblib.dump* (jako pliki *.pkl*), LSTM metodą *model.save* biblioteki Keras do formatu *.keras*. Obiekty skalujące (*scaler.pkl* dla modeli statycznych oraz *lstm_scalers.pkl* dla modelu sekwencyjnego) zapisywane są razem z modelami, ponieważ bez nich serwis predykcyjny nie byłby w stanie odtworzyć przestrzeni cech, w której modele zostały wytrenowane. Spójność między etapem trenowania a wnioskowaniem jest warunkiem koniecznym stabilnych predykcji, nawet drobna zmiana w rozkładach cech (różne parametry *mean* i *std* w skalowaniu) prowadziłaby do nieprzewidywalnego zachowania modeli w środowisku produkcyjnym.

W tym miejscu zamyka się zasadnicza część etapu implementacyjno-treningowego. Trzy modele, każdy reprezentujący inną rodzinę algorytmiczną i wykorzystujący dane w odmienny sposób, Random Forest i XGBoost jako klasyfikatory statyczne operujące na zagregowanym wektorze cech oraz LSTM jako klasyfikator sekwencyjny uczący się progresji zachowań w czasie, zostały wytrenowane, zwalidowane i zapisane w postaci artefaktów gotowych do eksploatacji. Ich porównanie ilościowe, ocena z perspektywy metryk biznesowych oraz interpretacja różnic pomiędzy ujęciem statycznym a dynamicznym są przedmiotem rozdziału 5.

Rozdział 5. Analiza wyników i ocena modeli

5.1. Metryki oceny jakości modeli klasyfikacyjnych

5.1.1. Dokładność, precyzja, czułość, F1-Score

5.1.2. Krzywa ROC i pole pod krzywą (AUC-ROC)

5.1.3. Macierz pomyłek i analiza błędów klasyfikacji

5.2. Wyniki modeli – zestawienie porównawcze

5.2.1. Wyniki modelu LSTM

5.2.2. Wyniki modelu Random Forest

5.2.3. Wyniki modelu XGBoost

5.3. Wzajemne porównanie modeli LSTM, Random Forest i XGBoost

5.4. Porównanie z klasycznymi metodami scoringowymi i istniejącymi rozwiązaniami

5.5. Interpretowalność modeli i ich przydatność decyzyjna

5.6. Dyskusja wyników i weryfikacja hipotez badawczych

Zakończenie

Bibliografia

Spis tabel

Spis rysunków i wykresów